

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

CSE-2340: Software Development 1

Accessibility Essentials

Week 2 · Class 2 · Landmarks, alt text, labels, heading order, keyboard, focus, contrast

Md Sadman Hafiz

Lecturer, Dept. of CSE, IIUC

Autumn 2026 | 50-minute laboratory class

Recall & Today's Outcome

? Recall from the last class

- What makes a group of radio buttons one group?
- Which attribute connects a label to an input?
- What does scope do on a th?
- Open your register.html now.

✓ By the end of today you can

- Explain who is blocked by an inaccessible page
- Check landmarks and heading order
- Write alt text using a simple decision rule
- Operate a whole page with the keyboard only
- Keep a visible focus indicator
- Check colour contrast and fix failures
- Run a Lighthouse accessibility audit

Who Is Blocked by a Bad Page?

Blind users

Hear the page through a screen reader. If the structure is wrong, the page is unusable.

Low vision users

Zoom to 200% or more, and need strong colour contrast.

Motor impairment

Cannot use a mouse. Everything must work with the keyboard.

Colour blindness

About 1 in 12 men. Cannot rely on red-versus-green alone.

Temporary and situational

A broken arm, bright sunlight, a slow connection, a noisy bus.

The curb-cut effect

Ramps were built for wheelchairs. Everyone with a suitcase uses them. Accessible pages are easier for everybody.

Accessibility in Four Questions

You do not need to memorise a standard. Ask these four questions about any page.

1 Can I see it?

Contrast is strong enough and text can be zoomed.

2 Can I hear it?

A screen reader announces headings, landmarks, labels and alt text.

3 Can I use it?

Everything works with the keyboard, and focus is always visible.

4 Can I understand it?

Labels, link text and error messages say what they mean.

Good news: semantic HTML answers most of these for free

If you used header, nav, main, footer, real headings, real labels and real buttons, you have already passed most checks.

Check 1: Landmarks

```
<body>
  <header>...</header>
  <nav aria-label="Main">...</nav>
  <main id="main-content">...</main>
  <aside>...</aside>
  <footer>...</footer>
</body>
```

What to look for in the audit

- A div id="header" instead of a header element
- More than one main on the page
- No main at all
- A nav that is really just a list of paragraphs

Why it matters

- A screen reader can list all landmarks and jump straight to one
- Without them the user must read the page from the top every time



How to check in 10 seconds

- Open DevTools
- Elements tab → Accessibility panel
- Read the accessibility tree
- Every region should have a name

Check 2: Heading Order

WRONG

```
<h1>Registration</h1>
<h4>Personal Info</h4>
<h2>Preferences</h2>
<h1>Routine</h1>
<h3>Notes</h3>
```

The rules

- Exactly one h1: the page title
- Never skip a level going down
- You may jump back up (h3 then h2)
- Never choose a heading level for its size - that is CSS

CORRECT

```
<h1>Registration</h1>
<h2>Personal Info</h2>
<h2>Preferences</h2>
<h2>Routine</h2>
<h3>Notes</h3>
```



Why users care

- Screen reader users press H to jump heading by heading
- Headings are the table of contents of your page
- A wrong order is like a book with chapter 1, then 4, then 2

Check 3: Alt Text Decision Rule

Ask one question: what job is this image doing on the page?

Informative

It carries information

`alt="Photo of Md Sadman at the IIUC campus"`

Decorative

It is only decoration

`alt=""` (empty, but still present)

Functional

It is inside a link or button

`alt="Search"` (describe the ACTION)

Text in image

It contains words

`alt="IIUC – 100 years of excellence"`

Never write: `alt="image"`, `alt="photo"`, `alt="picture1.jpg"`, or leave alt out completely.

Check 4: Labels and Form Names

```
<!-- WRONG: placeholder is not a label -->
<input type="text" name="fullname" placeholder="Name">

<!-- WRONG: text near the box is not connected -->
<span>Name</span>
<input type="text" id="n1">

<!-- CORRECT -->
<label for="fullname">Full Name</label>
<input type="text" id="fullname" name="fullname">

<!-- Groups need a name too -->
<fieldset>
  <legend>Preferred shift</legend>
  ...
</fieldset>
```

Why placeholder fails

- It disappears as soon as the user types
- Its contrast is usually too low
- Some screen readers ignore it



The 3-second test

- Click the label text
- If the cursor jumps into the box, it is connected
- If nothing happens, for and id do not match

Check 5: Keyboard Navigation

Tab	Move to the next control
------------	--------------------------

Shift + Tab	Move back
--------------------	-----------

Enter	Follow a link, press a button
--------------	-------------------------------

Space	Tick a checkbox, press a button
--------------	---------------------------------

Arrow keys	Move inside a radio group or select
-------------------	-------------------------------------

! Do this right now

- Put your mouse down
- Press Tab from the top of your page
- Can you reach and use every control?
- If not, your page has a real bug

Things that break the keyboard

- A div with a click handler instead of a button
- Custom dropdowns built from div tags
- tabindex values greater than 0
- Anything that only responds to a mouse

Check 6: Visible Focus

```
/* NEVER do this */
*:focus {
  outline: none;
}
```

```
/* If you must restyle it, replace it */
*:focus {
  outline: 3px solid #E39A18;
  outline-offset: 2px;
}
```



Why students remove it

- They find the default ring ugly
- A tutorial told them to
- Both are bad reasons. Restyle it, never delete it.

What focus is

- The ring the browser draws around the control you reached with Tab
- It is the keyboard user's mouse cursor
- Remove it and they are lost on the page



Skip link

- The first focusable item on the page
- Jumps past the nav to main
- Usually hidden until it receives focus
- You already added one last class

Check 7: Colour and Contrast

The numbers

- Normal text: 4.5 to 1
- Large text (24px+): 3 to 1
- Interface borders: 3 to 1

Fails

Light grey text on white

Yellow text on white

Passes

Dark navy text on white

How to check

- DevTools: click a colour swatch and read the contrast ratio
- Lighthouse reports every failure
- WebAIM contrast checker online

Never use colour alone

- "Fields in red are required" - invisible to a colour blind user
- Add a symbol, a word, or an icon as well



The fix

- Required fields: add the word Required or a * with an explanation
- Errors: add an icon and a text message, not just a red border

01

Live Audit: Fix the Broken Page

Open `week2-class2-inaccessible.html`. We find every problem and fix it together.

Running a Lighthouse Audit

1 Open the page with Live Server

2 Press F12 to open DevTools

3 Choose the Lighthouse tab

4 Tick only Accessibility

5 Click Analyze page load

6 Read each failure and open the affected element

! A score of 100 does NOT mean the page is accessible

- An automatic tool can only find about a third of real problems.
- It cannot tell you whether your alt text is meaningful or your heading order makes sense. Only a human can.

The Broken Page: What Is Wrong?

```
<body>
  <div id="header">
    <div class="title">IIUC Student Portal</div>
    <div class="menu">
      <span onclick="go('home')">Home</span>
      <span onclick="go('reg')">Register</span>
    </div>
  </div>

  <div id="content">
    <h3>Student Registration</h3>
    

    <form>
      <input type="text" name="fullname" placeholder="Name">
      <input type="text" name="sid" placeholder="Student ID">
      <p style="color:#c9cfd6">Fields in red are required</p>
      <div class="btn" onclick="submitForm()">Register</div>
    </form>
  </div>

  <div id="foot">Copyright 2026</div>
</body>
```

<!-- 1. not a heading, not a header -->

<!-- 2. cannot be reached by keyboard -->

<!-- 3. no h1, starts at h3 -->

<!-- 4. no alt attribute -->

<!-- 5. no label -->

<!-- 6. colour only, low contrast -->

<!-- 7. a div, not a button -->

<!-- 8. not a footer -->

The Fixed Page

```
<body>
  <a href="#main-content" class="skip-link">Skip to main content</a>

  <header>
    <h1>IIUC Student Portal</h1>
    <nav aria-label="Main navigation">
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="register.html" aria-current="page">Register</a></li>
      </ul>
    </nav>
  </header>

  <main id="main-content">
    <h2>Student Registration</h2>
    

    <form>
      <label for="fullname">Full Name (required)</label>
      <input type="text" id="fullname" name="fullname" required>

      <button type="submit">Register</button>
    </form>
  </main>

  <footer><p>&copy; 2026 IIUC Department of CSE</p></footer>
</body>
```

The Course Accessibility Checklist

Every mini project is checked against this list. Print it or keep it open.

- ✓ One h1, and heading levels never skip downwards
- ✓ Every image has an alt attribute (empty if decorative)
- ✓ Field groups use fieldset and legend
- ✓ Focus is always visible; outline is never set to none
- ✓ Meaning is never carried by colour alone
- ✓ header, nav, main and footer are present; exactly one main
- ✓ Every input has a label whose for matches its id
- ✓ Every control is reachable and usable with Tab alone
- ✓ Text contrast is at least 4.5 to 1
- ✓ Link text describes its destination; html has a lang attribute

Common Mistakes and Fixes

Using a div as a button

A div is not focusable and does not respond to Enter. Use `<button type="button">`.

alt attribute left out entirely

The screen reader reads the file name aloud. Use `alt=""` for decoration.

Placeholder used instead of a label

It vanishes when typing. Add a real label; keep the placeholder as a hint.

outline: none in the CSS

The keyboard user cannot see where they are. Replace the outline, never delete it.

Pale grey body text

Fails the 4.5 to 1 ratio. Darken the text or lighten the background.

Rule of thumb: if you are adding ARIA to make something work, first check whether a plain HTML element already does it.

Assignment - Week 2, Class 2

1 Part 1 - In-class demonstration

- Time: last 5 minutes of this class
- Task: fix at least four accessibility problems in the supplied broken page
- Run Lighthouse and show the score before and after
- Tab through the fixed page with no mouse
- Submission: show both scores on your screen before leaving
- Marks: 2 (checked on the spot)

2 Part 2 - Home task

- Audit your own register.html against the ten-point checklist
- Fix every failure you find
- Write audit-report.md: list each problem you found, the fix, and your Lighthouse score before and after
- Commit and push both files to GitHub
- Deadline: before Week 2, Class 3 · Marks: 3

The same checklist is used to mark Mini Project 1 next week.

48-50 MINUTES

Marking Rubric & Exit Check

RUBRIC - 5 MARKS TOTAL

Landmarks and heading order correct **1.0**

All images and form controls correctly named **1.5**

Page fully usable with the keyboard, focus visible **1.5**

Audit report explains each problem and its fix **1.0**

Exit question 1 - concept

Why is `alt=""` different from leaving the alt attribute out?

Exit question 2 - practical

Name two things that break when you replace a button element with a div.

Next class: CSS - application methods, selectors, text properties and units.